8051 Microcontroller

CJNE and the Five Addressing Modes

1. The Five Addressing Modes of the 8051

An addressing mode is simply the rule the CPU uses to figure out where an instruction's data actually lives.

In other words, addressing modes are various ways of accessing data (operands).

Data resides in

- a register,
- memory,
- or within the instruction itself.

The 8051 supports five addressing modes:

1. Immediate: the data is written directly into the instruction.
2. Register: the operand is a named register (A, R0–R7, DPH, DPL, ...).
3. Direct: the operand is an 8-bit address pointing into internal RAM or the SFRs.
4. Register Indirect: a register (R0, R1, or DPTR) holds the address of the data.
5. Indexed: an address is computed as a base plus an offset, used to read lookup tables in ROM.

1.1 Immediate Addressing (#)

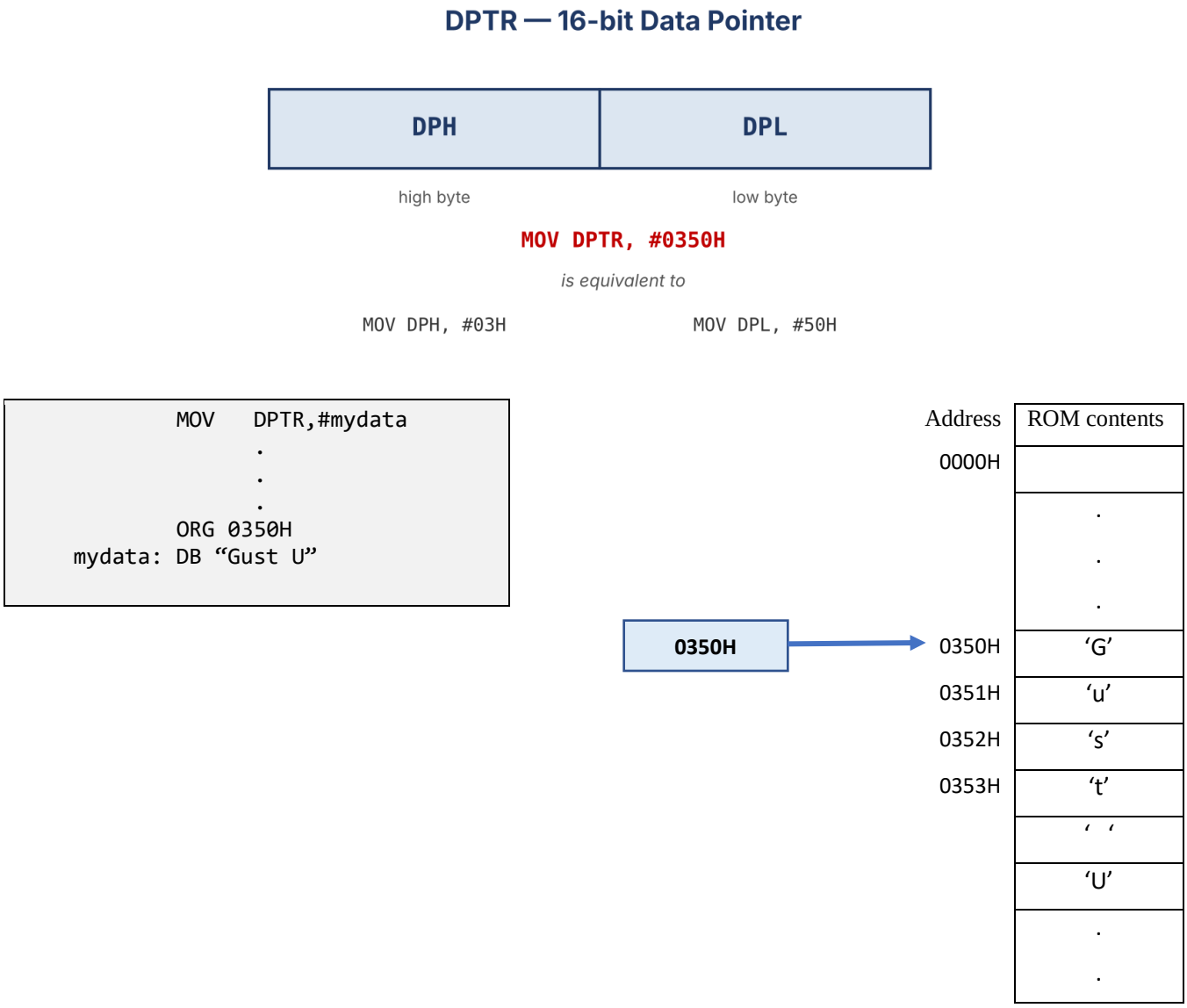
The value to be used is embedded directly in the instruction itself, signaled by the '#' symbol. It never refers to a memory location, the number right after # is the data.

```
        MOV   R4,#28H      ; R4 = 28H  (the literal value 28H)
```

```
        NUM1  EQU  50      ; NUM1 is a symbol standing for the

        MOV   A,#NUM1      ; immediate value 50 (decimal)
```

EQU and immediate data: A symbol defined with EQU (such as NUM1 EQU 50) is simply a name for a constant — wherever NUM1 is later used as #NUM1, the assembler substitutes the immediate value 50. It does not reserve a memory location.



1.2 Register Addressing (register name)

The operand is one of the CPU's named registers — the accumulator A, or the general-purpose registers R0–R7. No memory address is involved; the instruction's opcode itself identifies which register to use.

```
MOV   A, R4        ; copy the contents of R4 into A
MOV   A, DPH       ; copy the high byte of DPTR into A
```

1.3 Direct Addressing (00H to FFH)

The operand is an 8-bit address, written as a hex (or symbolic) number with no '#' and no '@'.

- 00H – 7FH → internal RAM
- 80H – FFH → the Special Function Registers (SFRs)

Example

```
MOV R7, 50H
```

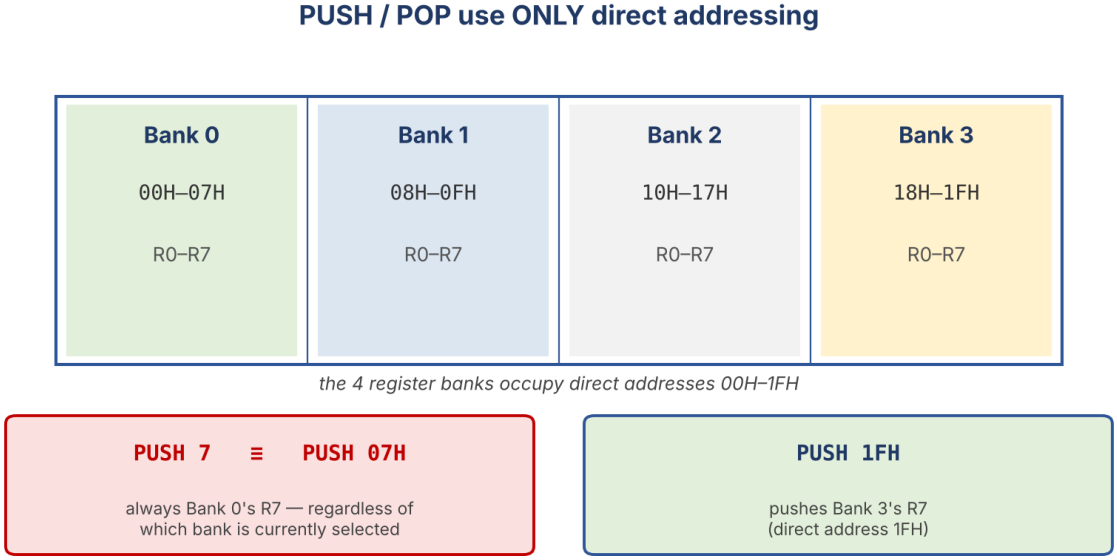
(direct addressing — read)

```
MOV 40H, A
```

(direct addressing — write)

Note: Push and pop use only direct addressing mode

PUSH and POP always use direct addressing for their operand — even when that operand looks like a register name such as R7. Internally, PUSH 7 is not “push whichever bank's R7 is currently selected”; it is always “push whatever byte sits at direct address 07H,” because 07H is where Bank 0's R7 physically lives.



The pitfall: If the program has switched to register bank 3, its R7 actually lives at direct address 1FH, not 07H. Writing PUSH 7 in that context silently pushes Bank 0's R7 instead — the wrong byte — with no error from the assembler. To push Bank 3's R7 correctly, you must write PUSH 1FH explicitly, naming the direct address that Bank 3's R7 actually occupies.

©2026 Dr. Fadi Alzhouri

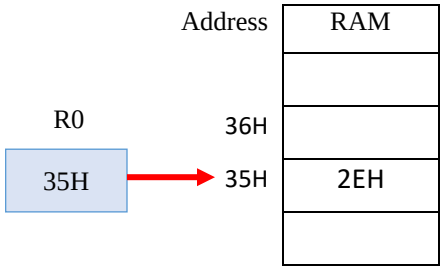
Exercise: Valid and Invalid Instructions

Instruction	Valid?	Why
PUSH R2		
PUSH 2		
PUSH 02H		
POP R4		
SETB 1FH		

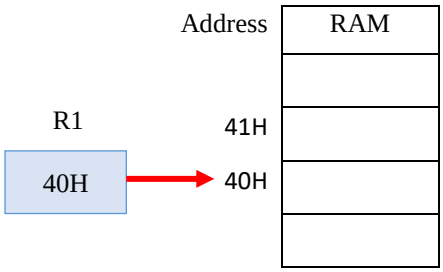
1.4 Register Indirect Addressing

Here, a register does not hold the data itself — it holds the address of the data, by using the '@' symbol. Only two registers may be used as 8-bit pointers this way: R0 and R1 (never R2–R7). A third option, @DPTR, uses the 16-bit DPTR as a pointer into external memory.

```
MOV R0, #35H
MOV A,@R0
```




```
MOV R1, #40H
MOV A,#72H
MOV @R1,A
```



- R0 or R1 as an 8-bit pointer can address internal RAM (00H–7FH) or, for external memory, a full 8-bit range (00H–FFH).
- @DPTR, being 16-bit, can address the entire external memory space: 0000H–FFFFH.

Restriction: MOV @R0,@R1 is invalid, an instruction cannot have both its source and its destination use register-indirect addressing at the same time. One side of a MOV may be indirect, but not both.

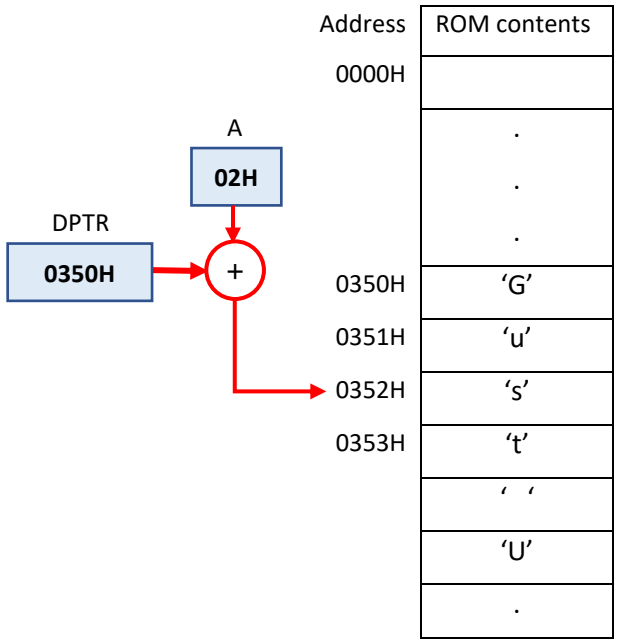
1.5 Indexed Addressing (@ Reg + Reg)

Indexed addressing is used specifically with the MOVC instruction to read constant data (lookup tables) stored in code memory (ROM). The effective address is computed as a base address (held in DPTR) plus an offset (held in the accumulator, A):

MOVC A, @ A + DPTR

Example

```
MOV DPTR,#350H
MOV A,#2
MOVC A,@A+DPTR
```



Why indexed addressing matters: This allows the CPU to access memory locations dynamically by adding an offset to a base address. In other words, it enables us to easily read a lookup table.

1.6 Summary Table

Mode	Symbol	Operand Is…	Example
Immediate	#	a literal value	`MOV A,#28H`
Register	(name)	a named register	`MOV A,R4`
Direct	(address)	00H - FFH	`MOV A,50H`
Register Indirect	@Ri / @DPTR	a register holding an address	`MOV A,@R0`
Indexed	@A+DPTR	base + offset, in ROM	`MOVC A,@A+DPTR`

2. The CJNE Instruction

CJNE stands for “Compare and Jump if Not Equal.” It compares two operands and jumps to a **label** if they are different. It falls through to the next instruction only when the two operands are equal.

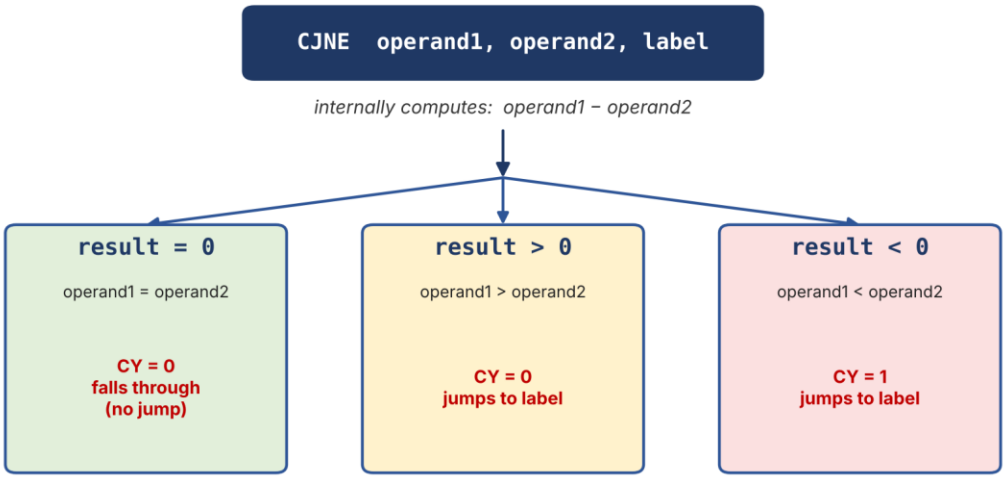
2.1 The Four Operand Forms

CJNE can compare the accumulator or a register/memory location against an immediate value, a direct address, or an indirectly-addressed byte:

```
CJNE A, #data, label
CJNE A, direct, label
CJNE Rn, #data, label
CJNE @Ri, #data, label
```

2.2 How the Comparison Works

Internally, CJNE performs the subtraction (operand1 − operand2) and uses the result to set the Carry flag (CY), exactly the way CY is set after a normal subtraction. The jump decision and the value of CY both depend on the sign of that result:



After the jump, JC / JNC can then distinguish “greater than” from “less than.”

Why this matters: CJNE only tells you whether the operands are equal or not — by itself it does not tell you which one is larger. Because CY ends up set only when operand1 < operand2, a JC (jump if carry) or JNC (jump if no carry) placed right after the CJNE can test CY to tell “greater than” apart from “less than,” giving you a full three-way comparison (=, <, >) from two instructions.

©2026 Dr. Fadi Alzhouri

2.3 Example

Suppose R3 holds the value to be checked against 25H:

```
        MOV   R3, #30H
        CJNE  R3, #25H, NOTEQ
                                ;  R3 = 25H


        SJMP DONE
NOTEQ:  JC    LESS          ; CY = 1  →  R3 < 25H
                               ; R3 is greater than 25H


        SJMP DONE
LESS:                        ; R3 is less than 25H


DONE:
```